



Lab 6

Combinational Logic (SystemVerilog)

Prof. Chih-Cheng Tseng (曾志成)

cctseng@mail.ntou.edu.tw

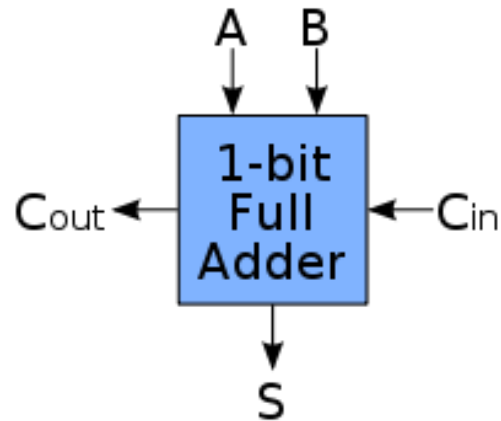
Contents of this slide are provided by NTOU VLSI Lab601

Objectives

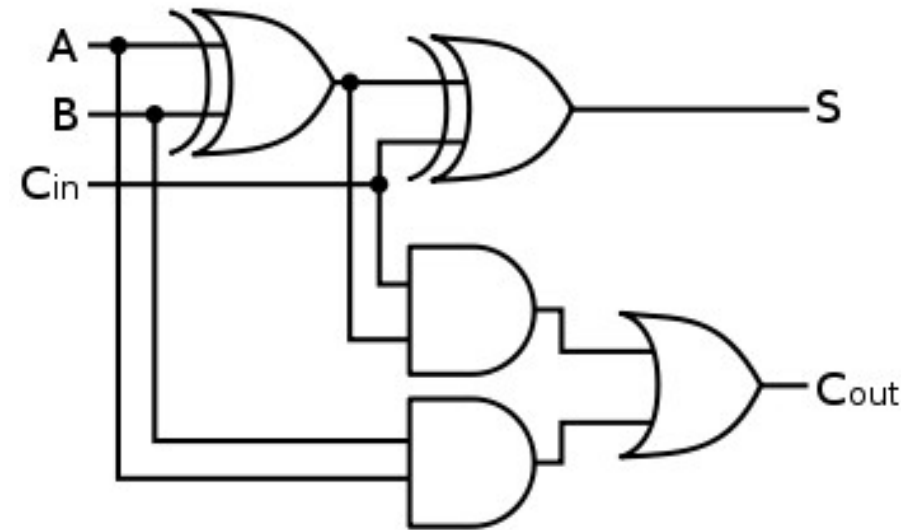
- Write a full adder using SystemVerilog.
- Write a decoder using SystemVerilog.
- Write a multiplexer using SystemVerilog.
- Write a 4-bit adder using SystemVerilog.
- Write a 4-bit adder/subtractor using SystemVerilog

Full Adder

- Symbol



- Internal structure



Full Adder - HDL Code

```

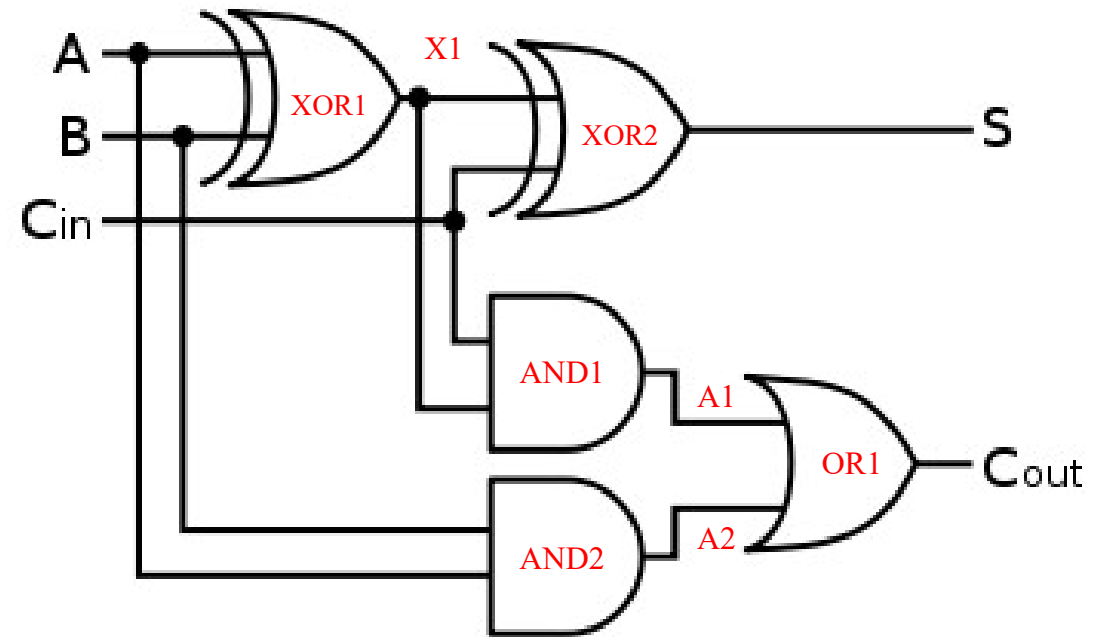
module FA_1B(
    input A,
    input B,
    input Cin,
    output logic S,
    output logic Cout
);

    logic X1;
    logic A1, A2;

    xor XOR1(X1, A, B);
    xor XOR2(S, X1, Cin);
    and AND1(A1, X1, Cin);
    and AND2(A2, A, B);
    or OR1(Cout, A1, A2);

endmodule

```



Full Adder - Test Bench

```

module tb_FA_1B;

    logic A, B, Cin;
    logic Cout, S;

    FA_1B u_FA_1B(
        .A(A),
        .B(B),
        .Cin(Cin),
        .Cout(Cout),
        .S(S)
    );

    initial
    begin
        A = 0; B = 0; Cin = 0;
        #10 A = 0; B = 0; Cin = 1;
        #10 A = 0; B = 1; Cin = 0;
        #10 A = 0; B = 1; Cin = 1;
        #10 A = 1; B = 0; Cin = 0;
        #10 A = 1; B = 0; Cin = 1;
        #10 A = 1; B = 1; Cin = 0;
        #10 A = 1; B = 1; Cin = 1;
        #10 $stop;
    end

    initial
    begin
        $monitor($time, " A=%b, B=%b, Cin=%b, S=%b, Cout=%b", A, B, Cin, S, Cout);
    end

endmodule

```

Full Adder - Do file (.do) Preparation

compile.do

```
#vlib work

# -----
vlog ../tb/tb_FA_1B.sv
vlog ../../design/FA_1B.sv

# -----
```

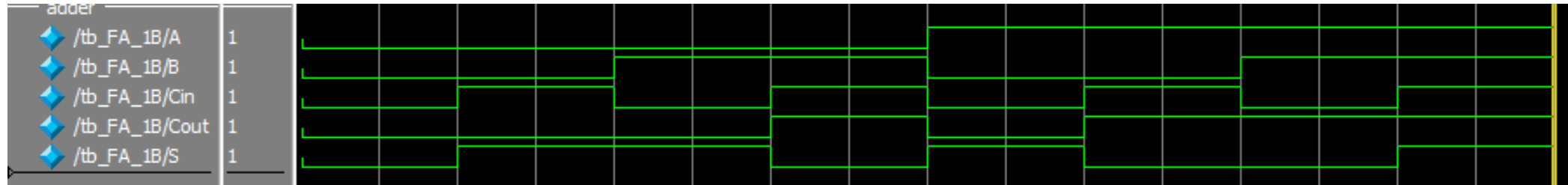
sim.do

```
vsim -voptargs=+acc work.tb_FA_1B
view structure wave signals

do wave.do

log -r *
run -all
```

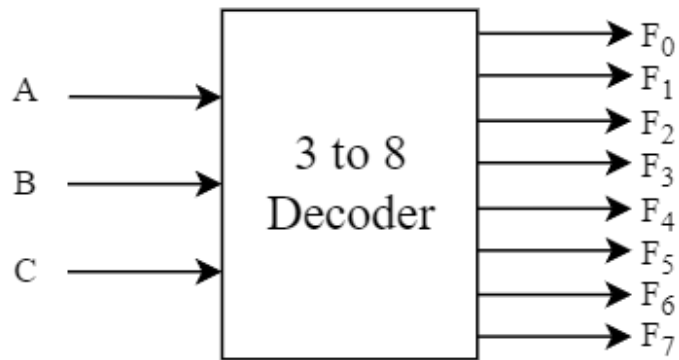
Full Adder - MODELSIM Simulation



```
#           0 A=0, B=0, Cin=0, S=0, Cout=0
#           10 A=0, B=0, Cin=1, S=1, Cout=0
#           20 A=0, B=1, Cin=0, S=1, Cout=0
#           30 A=0, B=1, Cin=1, S=0, Cout=1
#           40 A=1, B=0, Cin=0, S=1, Cout=0
#           50 A=1, B=0, Cin=1, S=0, Cout=1
#           60 A=1, B=1, Cin=0, S=0, Cout=1
#           70 A=1, B=1, Cin=1, S=1, Cout=1
```

Lab 1 - 3 to 8 Decoder

- Input: A, B, C
- Output: $F_0 \sim F_7$
- Write the program using gate level
- Run Modelsim simulation



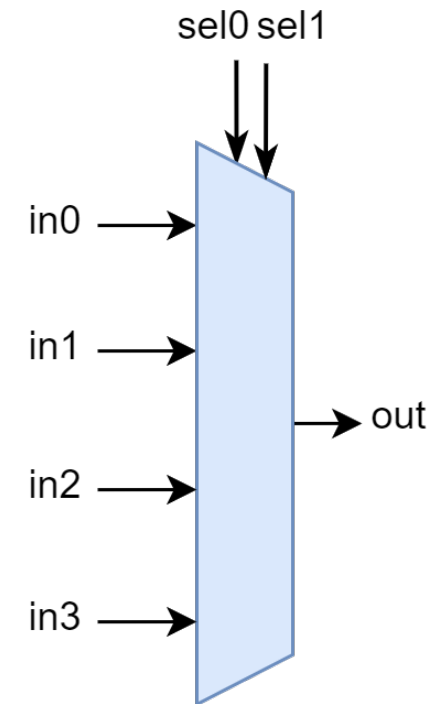
A	B	C	F_0	F_1	F_2	F_3	F_4	F_5	F_6	F_7
0	0	0	1	0	0	0	0	0	0	0
0	0	1	0	1	0	0	0	0	0	0
0	1	0	0	0	1	0	0	0	0	0
0	1	1	0	0	0	1	0	0	0	0
1	0	0	0	0	0	0	1	0	0	0
1	0	1	0	0	0	0	0	1	0	0
1	1	0	0	0	0	0	0	0	1	0
1	1	1	0	0	0	0	0	0	0	1

Lab 2 - 4 to 1 Multiplexer

- Input: sel0, sel1, in0 ~ in3
- Output: out
- Write the program using gate level
- Run Modelsim simulation

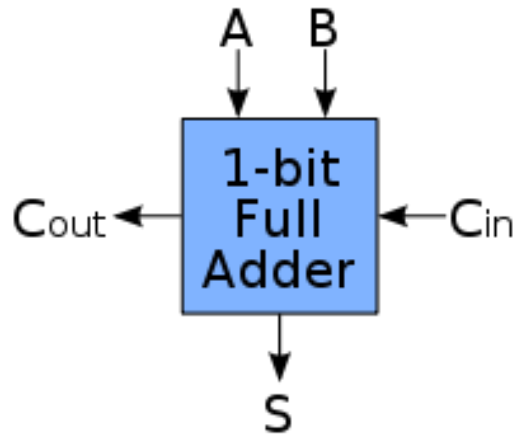
sel ₁	sel ₀	out
0	0	in0
0	1	in1
1	0	in2
1	1	in3

Signal	Value
in0	0
in1	1
in2	1
in3	0



Lab 3 - Full Adder

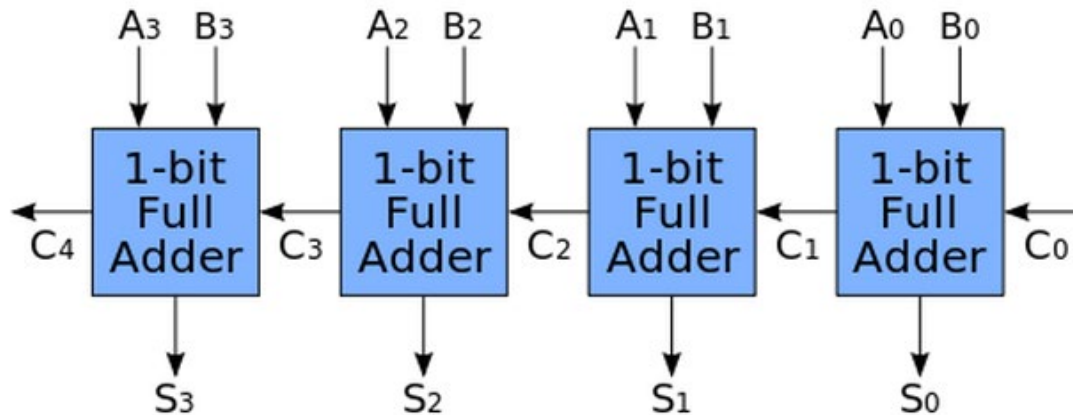
- Input: A, B, Cin
- Output: Cout, S
- Write the program using gate level
- Run Modelsim simulation



Test Bench		
A	B	Cin
0	0	0
0	0	1
0	1	0
0	1	1
1	0	0
1	0	1
1	1	0
1	1	1

Lab 4 - 4 Bits Adder

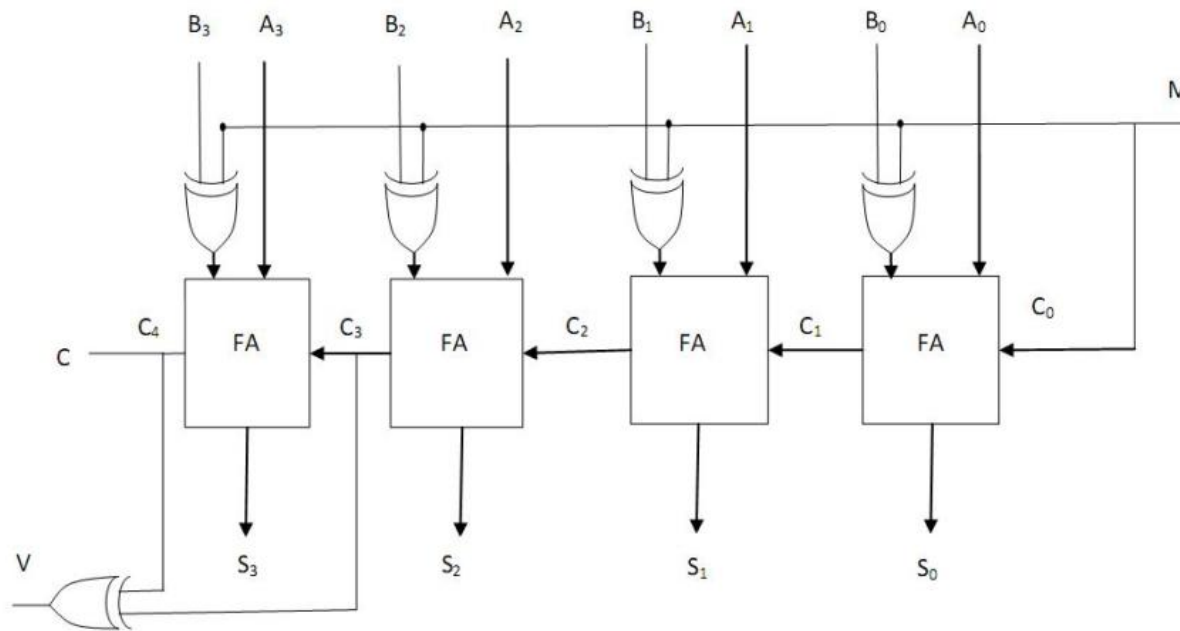
- Input: $A_3 \sim A_0$, $B_3 \sim B_0$, C_0
- Output: C_4 , $A_3 \sim A_0$
- Write the program using gate level
- Run Modelsim simulation



Test Bench		
$A_{3\sim0}$	$B_{3\sim0}$	C_0
0111	0110	0
1000	1001	0
1100	1000	1
0101	1010	1
0000	0001	1

Bonus - 4-Bit Adder/Subtractor

- Input: $A_3 \sim A_0$, $B_3 \sim B_0$, M
- Output: C , $S_3 \sim S_0$, V
- Write the program using gate level
- Run Modelsim simulation



Test Bench		
$A_{3\sim0}$	$B_{3\sim0}$	M
0111	0110	0
1000	1001	0
1100	1000	1
0101	1010	1
0000	0001	1